

claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf_b531eaeef-39c\agent-  
a0be0c03744ec2bc3.jsonl`  
- SHA-256: `8d6ae3f8dcbd1fa3e0ce5f886f3d54db4f7658f7e2ee0baaa1499de0f04d36df`  
- Source modified: `2026-06-19T16:58:07+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_b531eaeef-39c`  
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`
```

Transcript

1. user (2026-06-19T16:57:07.032Z)

You are verifying claims for a design doc; the doc must only assert what the code supports. Verify how the single shuttle trajectory becomes rallies in PRODUCTION: trace `parse_trajectory_csv ? trajectory_to_action_windows` (backend/pipeline/detectors/tracknet_runner.py) and the segmenter that drives it (backend/pipeline/segmenters/trajectory_hybrid.py and/or wasb_hybrid). Identify the EXACT seam where a multi-track selection layer would inject (does the windower take a single point-stream? could it take a selected track or a union of per-track windows?). Cite lines. VERIFIED/REFUTED/PARTIAL.

Read the actual files (do not guess). Return the structured verdict.

2. assistant (2026-06-19T16:57:10.129Z)

I'll trace the production path from the trajectory CSV to rallies. Let me start by reading the two key files.

3. user (2026-06-19T16:57:11.568Z)

```
1      """Shared engine for trajectory-substrate hybrid segmenters.  
2  
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a  
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is  
5      now the single home for the whole pipeline shape:  
6  
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)  
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking  
9  
10     A concrete subclass supplies only its identity and runner wiring:  
11     - ``family`` ? config section under ``indexing.*``, output subdir, and  
12       the ``<family>-hybrid-<model>`` detector tag in metadata.  
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").  
14     - ``name`` / ``description`` properties (the registry contract).  
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific  
16       detection_params extras such as weights_path).  
17  
18     This base is deliberately NOT registered; only concrete subclasses register.  
19     """  
20     import os  
21     import time  
22     import logging  
23     import concurrent.futures  
24     from typing import Any, Dict, List, Optional, Tuple  
25  
26     import cv2
```

```

27
28 from backend.pipeline.segmenters.base import BasePlaySegmenter
29 from backend.utils.video import get_video_duration, extract_video_chunk
30 from backend.sports import sport_registry
31 from backend.providers import provider_registry
32 from backend.pipeline.detectors.base import DetectorRunner
33
34 logger = logging.getLogger(__name__)
35
36
37 def _fmt_ts(seconds: float) -> str:
38     seconds = max(0, int(seconds))
39     h, rem = divmod(seconds, 3600)
40     m, s = divmod(rem, 60)
41     return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44 class TrajectoryHybridSegmenter(BasePlaySegmenter):
45     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46 boundaries."""
47
48     #: config section under `indexing` (velocity_threshold lives there), the
49     #: output subdir for trajectories, and the metadata detector-tag prefix.
50     family: str = ""
51     #: human-readable label used in log lines.
52     display_label: str = ""
53
54     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56         """Return (runner, detector-specific detection_params extras)."""
57         raise NotImplementedError
58
59     @staticmethod
60     def _probe_fps_width(video_path: str) -> tuple:
61         """Return (fps, frame_width) for a video, with sensible fallbacks."""
62         cap = cv2.VideoCapture(video_path)
63         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65         cap.release()
66         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68     @staticmethod
69     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70         """Provider-reported exchange count, else a duration-based estimate.
71
72         One canonical implementation ? the twins had drifted (wasb relied on
73         ``int(None)`` raising into the fallback; same result, by accident).
74         """
75         shot_count = segment.get("shuttle_exchange_count")
76         if shot_count is None:
77             return max(3, int(duration / 0.8))
78         try:
79             return int(shot_count)
80         except (ValueError, TypeError):
81             return max(3, int(duration / 0.8))
82
83     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84     -----
85     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86 frame_width: float, video_path: str,
87 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89 BASE
90 returns exactly the 4 motion keys, so the trajectory twins persist byte-
91 identical

```

```

87         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89         return {
90             "peak_velocity": cw["peak_velocity"],
91             "mean_velocity": cw["mean_velocity"],
92             "active_frame_count": cw["active_frame_count"],
93             "track_id_count": cw["track_id_count"],
94         }
95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                             window_stats: List[Dict[str, Any]],
98                             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                       player_pool: Optional[List[str]] = None,
108                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
123         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
124         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
125         # measure it against the Gemini path, or to avoid the cloud entirely). With
126         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
127         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
128         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
129         if skip_ai:
130             model_name = "local-noai"
131             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
132                         "be emitted as rallies; no %s handoff, no API key needed.",
133                         label, provider_name)
134         else:
135             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
136             api_key_env_var = f"{provider_name.upper()}_API_KEY"
137             api_key = os.environ.get(api_key_env_var)
138             if not api_key:
139                 logger.error(
140                     f"{api_key_env_var} environment variable is not set! "
141                     f"To run the {provider_name} handoff, set your API key. "
142                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")

```

```

143         )
144         return []
145     try:
146         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
147     except KeyError as e:
148         logger.error(str(e))
149         return []
150
151     video_duration = get_video_duration(video_path)
152     if video_duration <= 0:
153         logger.error("Invalid video duration.")
154         return []
155     fps, frame_width = self._probe_fps_width(video_path)
156
157     # --- Runner config + windowing thresholds ---
158     runner, runner_params = self._build_runner(idx_cfg)
159
160     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
163     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
164     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
165     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
166     window_hard_cap = idx_cfg.get("window_hard_cap", False)
167     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
168     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
169     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
170     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
171     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
172     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
173     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
174     log_candidates = idx_cfg.get("log_yolo_candidates", True)
175     store_candidates = idx_cfg.get("store_yolo_candidates", True)
176
177     detection_params = {
178         "detector": self.name,
179         "velocity_thresh": velocity_thresh,
180         "merge_gap": merge_gap,
181         "min_action_window_duration": min_window_duration,
182         **runner_params,
183     }
184
185     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
186     ok, msg = runner.healthcheck()
187     if not ok:
188         logger.error("%s WSL environment not ready: %s", label, msg)
189         return []
190     logger.info("%s WSL env OK: %s", label, msg)
191
192     # --- Phase 2: trajectory -> candidate rally windows ---
193     # Trajectory detectors process the whole video in one pass (they carry
194     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
195     t_start = time.time()
196     out_dir = os.path.join("output", self.family)
197     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
198     if not csv_path:
199         logger.error("%s produced no trajectory; aborting.", label)
200         return []
201
202     points = runner.parse_trajectory_csv(csv_path)
203     logger.info("Parsed %d trajectory points (%d visible).",
204                 len(points), sum(1 for p in points if p.visible))
205

```

```

206         all_candidate_windows = runner.trajectory_to_action_windows(
207             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
208             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
209             min_window_duration=min_window_duration, chunk_index=0,
210             max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
211             window_hard_cap=window_hard_cap,
212             window_inactivity_end=window_inactivity_end,
213             inactivity_rally_thresh=inactivity_rally_thresh,
214             inactivity_min_density=inactivity_min_density,
215             window_blob_fallback=window_blob_fallback,
216             window_blob_factor=window_blob_factor,
217         )
218         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
219                     label, time.time() - t_start, len(all_candidate_windows))
220
221         if log_candidates:
222             for cw in all_candidate_windows:
223                 logger.info(f"[{label} rally]
224 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
225                             f"({cw['end'] - cw['start']:.1f}s) |
226 frames={cw['active_frame_count']} "
227                             f"peak_v={cw['peak_velocity']:.3f}
228 mean_v={cw['mean_velocity']:.3f}")
229
230         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
231         fusion
232         # segmenter adds net_crossings + person-cue features. Computed once, reused for
233         both
234         # persistence and the handoff gate below.
235         window_stats = [
236             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
237             idx_cfg)
238             for cw in all_candidate_windows
239         ]
240
241         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
242         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
243         if store_candidates:
244             for i, cw in enumerate(all_candidate_windows):
245                 candidate_ids[i] = self.db.add_candidate_segment(
246                     video_id, detector=self.name,
247                     start_time=cw["start"], end_time=cw["end"],
248                     chunk_index=cw["chunk_index"], confidence=min(1.0,
249                     cw["peak_velocity"]),
250                     stats=window_stats[i], detection_params=detection_params,
251                 )
252
253         if not all_candidate_windows:
254             return []
255
256         # --- Phase 3: AI provider handoff over padded candidate clips ---
257         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
258         Persisted
259         # candidates above stay the full superset ? only the served play_segments are
260         gated.
261         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
262         idx_cfg)
263
264         ai_segments: List[dict] = []
265         if skip_ai:
266             # Fully-local: the selected candidate windows ARE the rallies ? emit them
267             verbatim
268             # (no clip extraction, no upload). Boundaries are the local detector's; shot
269             count
270             # falls back to the duration-based estimate (no AI exchange count).

```

```

259         ai_segments = [
260             {
261                 "start_time": all_candidate_windows[wi]["start"],
262                 "end_time": all_candidate_windows[wi]["end"],
263                 "shuttle_exchange_count": None,
264                 "shot_count_confidence": "LocalNoAI",
265                 "_candidate_id": candidate_ids[wi],
266             }
267             for wi in handoff_indices
268         ]
269         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
270                     "(no AI handoff).", len(ai_segments))
271     else:
272         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
273         os.makedirs(handoff_dir, exist_ok=True)
274         logger.info("Phase 3: %s validation on %d candidate windows...",
275                     provider_name, len(handoff_indices))
276
277         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
278             w_start = max(0, window["start"] - handoff_padding)
279             w_end = min(video_duration, window["end"] + handoff_padding)
280             w_dur = w_end - w_start
281             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
282             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
283                 return []
284             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
285             segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
286                                                  label=f"Handoff {wi+1}")
287             valid = []
288             for seg in segments:
289                 try:
290                     seg["start_time"] = float(seg["start_time"]) + w_start
291                     seg["end_time"] = float(seg["end_time"]) + w_start
292                     seg["_candidate_id"] = candidate_ids[wi]
293                     valid.append(seg)
294                 except (ValueError, TypeError, KeyError) as e:
295                     logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
296             try:
297                 os.remove(w_path)
298             except OSError:
299                 pass
300             return valid
301
302         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
303             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
304                         for wi in handoff_indices]
305             for future in futures:
306                 ai_segments.extend(future.result())
307             try:
308                 os.rmdir(handoff_dir)
309             except OSError:
310                 pass
311             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
312
313         # --- Phase 4: DB population + candidate linking ---
314         segments_data = []
315         for segment in ai_segments:
316             try:

```

```

317         start = float(segment["start_time"])
318         end = float(segment["end_time"])
319         dur = end - start
320         metadata = {
321             "shot_count": self._shot_count_from(segment, dur),
322             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
323             "detector": f"{self.family}-hybrid-{model_name}",
324         }
325         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
326         if segment_id:
327             candidate_id = segment.get("_candidate_id")
328             if candidate_id is not None:
329                 self.db.link_candidate_to_segment(candidate_id, segment_id)
330             segments_data.append({
331                 "id": segment_id, "start_time": start, "end_time": end,
332                 "duration": dur, "metadata": metadata,
333             })
334         except Exception as e:
335             logger.warning("Skipping invalid segment struct: %s", e)
336
337     return segments_data
338

```

4. user (2026-06-19T16:57:11.745Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7            stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"

```

```

37         weights_path: str = ""                                # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                               # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                         # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                                # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                              # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                              # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78
79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float
84         y: float
85
86
87     class TrackNetRunner(WslCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90         Implements the common DetectorRunner contract so a future Linux-native or
91         alternative trajectory runner can be substituted without touching the hybrids.
92         """
93
94         def __init__(self, cfg: TrackNetConfig):
95             self.cfg = cfg

```



```

96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n\""
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "wsl` not found ? is this running on Windows with WSL2
112             installed?"
113         except subprocess.TimeoutExpired:
114             return False, "WSL healthcheck timed out."
115         out = (res.stdout or "").strip()
116         if res.returncode != 0:
117             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118         return True, out
119
120     # ----- inference
121
122     def run_predict(self, video_win_path: str, output_win_dir: str,
123                    video_id: Optional[str] = None) -> Optional[str]:
124         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125         None.
126
127         ``output_win_dir`` is a Windows directory (under the repo's output/) that
128         WSL writes into via its /mnt view, so the Windows process can read the CSV
129         back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131         a filename cannot collide on the output CSV.
132         """
133         if not self.cfg.weights_path:
134             logger.error(
135                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136                 "in config.json to the WSL path of your trained TrackNetV4 weights."
137             )
138             return None
139
140         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
141         # relative paths through unchanged, so WSL would resolve them against the
142         # predict.py cwd instead of the repo (same bug class as wasb_runner).
143         output_win_dir = os.path.abspath(output_win_dir)
144         os.makedirs(output_win_dir, exist_ok=True)
145         # Normalize separators so basename/splitext work when tests (or collector)
146         # pass Windows-style paths on a Linux host.
147         video_win_path = str(video_win_path).replace("\\", "/")
148         base = os.path.basename(video_win_path)
149         stem, ext = os.path.splitext(base)
150
151         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
152         if ext.lower() not in (".mp4", ".avi"):
153             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
154             return None
155
156         wsl_out = to_wsl_mnt_path(output_win_dir)
157         # Namespace by video_id so two same-filename videos don't collide on the CSV.
158         # predict.py derives its output name from the (staged) video stem, so staging
159         # under the namespaced name propagates into "<key>_predict.csv".
160         key = f"{stem}_{video_id[:12]}" if video_id else stem

```

```

158         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         # We cannot rename its output, so fall back to the un-namespaced name.
169         wsl_video = to_wsl_mnt_path(video_win_path)
170         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
171
172     if self.cfg.min_free_gb > 0:
173         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
174         if free is not None and free < self.cfg.min_free_gb:
175             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                           "consider running tools/cleanup_caches.py.",
177                           free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179     cmd = (
180         f"conda activate {self.cfg.conda_env} && "
181         f"cd {self.cfg.repo_dir}/src && "
182         f"python predict.py "
183         f"--video_path {wsl_tilde_quote(wsl_video)} "
184         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185         f"--output_dir {wsl_tilde_quote(wsl_out)} "
186         f"--queue_length {self.cfg.queue_length}"
187     )
188     logger.info("Running TrackNet inference on %s ...", base)
189     try:
190         res = self._wsl_bash(cmd)
191     except subprocess.TimeoutExpired:
192         logger.error("TrackNet inference timed out after %ss.",
193                     self.cfg.timeout_sec)
194         return None
195
196     if res.returncode != 0:
197         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
198                     res.stdout)[-2000:])
199         return None
200
201     if not os.path.exists(expected_csv_win):
202         logger.error("TrackNet finished but expected CSV not found at %s",
203                     expected_csv_win)
204         return None
205     logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
206
207     # Success ? the CSV is the durable output; drop the staged video copy (a pure,
208     # regenerable intermediate). On earlier failure we return above without
209     cleaning.
210     if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
211         logger.info("Cache hygiene: removing staged video for %s "
212                     "(set indexing.tracknet.keep_frames=true to retain).", key)
213         self._wsl_rm_rf([wsl_video])
214     return expected_csv_win
215
216 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
217     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
218
219     Returns the WSL path to the staged copy, or None on failure.
220     """
221     src_mnt = to_wsl_mnt_path(video_win_path)

```

```

218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
224         {wsl_tilde_quote(dest)} && echo OK"
225         try:
226             res = self._wsl_bash(cmd)
227         except subprocess.TimeoutExpired:
228             logger.error("Staging video into WSL timed out.")
229             return None
230         if res.returncode != 0 or "OK" not in (res.stdout or ""):
231             logger.error("Failed to stage video into WSL: %s", (res.stderr or
232 res.stdout).strip())
233             return None
234             return dest
235
236 # ----- CSV -> windows
237
238 @staticmethod
239 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
240     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
241     points: List[TrajectoryPoint] = []
242     with open(csv_win_path, newline="") as f:
243         reader = csv.DictReader(f)
244         for row in reader:
245             try:
246                 points.append(TrajectoryPoint(
247                     frame=int(row["Frame"]),
248                     visible=int(row["Visibility"]) == 1,
249                     x=float(row["X"]),
250                     y=float(row["Y"]),
251                 ))
252             except (KeyError, ValueError):
253                 continue
254     points.sort(key=lambda p: p.frame)
255     return points
256
257 @staticmethod
258 def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
259 velocity_thresh: float) -> List[Tuple[int, float]]:
260     """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
261 flight.
262 Computes each visible point's normalised velocity from the last *visible* point;
263 an absent shuttle breaks the trajectory. A frame qualifies when its velocity
264 exceeds
265 ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
266 defaulted by
267 the caller so identical floats feed the velocity math."""
268     active = [] # (frame_idx, velocity)
269     prev: Optional[TrajectoryPoint] = None
270     for p in points:
271         if not p.visible:
272             prev = None # break the trajectory; absent shuttle = not in flight
273             continue
274         if prev is not None:
275             dframes = p.frame - prev.frame
276             if dframes > 0:
277                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
278                 velocity = (dist / frame_width) * (fps / dframes)
279                 if velocity > velocity_thresh:
280                     active.append((p.frame, velocity))
281         prev = p

```

```

278         return active
279
280     @staticmethod
281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,
294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
309         against
310         the *over-merge* failure where "shuttle moving" is conflated with "rally in
311         progress"
312         and a single window swallows several rallies + the dead time between them. When
313         set, any
314         window longer than this is recursively SPLIT at its largest internal inactivity
315         gap (the
316         longest stretch with no in-flight shuttle ? the most likely rally boundary),
317         provided
318         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
319         fix from
320         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
321         recall cannot
322         drop, and it is config-gated default-OFF until measured on the golden set.
323
324         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
325         an
326         over-long window has NO internal inactivity gap to split on (a continuously-
327         active
328         trajectory ? the shuttle never stops moving, e.g. the real-footage
329         ``mahadevpura-1``
330         blob where the gap-splitter alone left a single 174 s window), it is recursively
331         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
332         cuts
333         (never merges), so recall cannot drop; gated default-OFF until measured.
334
335         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
336         unlike
337         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
338         every gap-less
339         over-cap window and over-segments normal clips), this runs AFTER the R4
340         inactivity trim and
341         force-chops ONLY a genuine *blob* ? a group still longer than
342         ``window_blob_factor`` x

```

```

328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         """
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
338             frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]
367         groups.append(group)
368
369         if max_window_duration > 0:
370             groups = TrackNetRunner._split_overlong_groups(
371                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                 hard_cap=window_hard_cap)
373
374         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
378         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379         if window_inactivity_end > 0:
380             tw = int(window_inactivity_end * fps)

```

```

381         groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                                                         inactivity_rally_thresh) for g
in groups]
383
384         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388         forced_flags = [False] * len(groups)
389         if window_blob_fallback and max_window_duration > 0:
390             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391                 groups, int(max_window_duration * fps), window_blob_factor)
392
393         windows = []
394         for g, forced in zip(groups, forced_flags):
395             w = _finalize(g)
396             if forced:
397                 w["forced_cut"] = True
398             windows.append(w)
399         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401     @staticmethod
402     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
404         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
405         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
406         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
407         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
408         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
409         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
410         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
411         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
412         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
413         if trail_frames <= 0 or len(group) < 2:
414             return group
415         frames = [f for f, _ in group]
416         fast = [1 if v > rally_thresh else 0 for _, v in group]
417         lo = 0
418         fast_sum = 0
419         last_dense = -1
420         for i in range(len(frames)):
421             fast_sum += fast[i]
422             while frames[lo] <= frames[i] - trail_frames:
423                 fast_sum -= fast[lo]
424                 lo += 1
425             win_count = i - lo + 1
426             if win_count > 0 and fast_sum / win_count >= min_density:
427                 last_dense = i
428         return group[: last_dense + 1] if last_dense >= 0 else group
429
430     @staticmethod
431     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:

```

```

432         """Index of the active frame nearest the group's temporal midpoint (used to chop
a
433         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
434         mid = (group[0][0] + group[-1][0]) / 2.0
435         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
436         return split_i
437
438     @staticmethod
439     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
440                               min_split_frames: int,
441                               hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
442         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
443         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
444         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
445         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
446         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
447         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
448         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
449         if max_win_frames <= 0:
450             return groups
451         out: List[List[Tuple[int, float]]] = []
452         work = list(groups)
453         while work:
454             g = work.pop()
455             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
456                 out.append(g)
457                 continue
458             best_i, best_gap = -1, -1
459             for i in range(1, len(g)):
460                 gap = g[i][0] - g[i - 1][0]
461                 if gap > best_gap:
462                     best_gap, best_i = gap, i
463             if best_i < 0 or best_gap < min_split_frames:
464                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
465                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
466                 if hard_cap:
467                     split_i = TrackNetRunner._midpoint_split_index(g)
468                     work.append(g[:split_i])
469                     work.append(g[split_i:])
470                 else:
471                     out.append(g)
472                     continue
473                 work.append(g[:best_i])
474                 work.append(g[best_i:])
475             out.sort(key=lambda grp: grp[0][0])
476             return out
477
478     @staticmethod
479     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                             blob_factor: float = 4.0
481                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483         inactivity trim have each had their chance, a group still longer than

```

```

``blob_factor`` x
484     ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485     signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486     11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
487     is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
488     midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489     degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490     rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491     can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
492     that piece came from a forced chop. Iterative worklist to bound stack depth.""
493     if max_win_frames <= 0:
494         return groups, [False] * len(groups)
495     blob_frames = max_win_frames * max(1.0, blob_factor)
496     out: List[List[Tuple[int, float]]] = []
497     flags: List[bool] = []
498     forced = 0
499     for g in groups:
500         if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501             out.append(g)
502             flags.append(False)
503             continue
504     pieces: List[List[Tuple[int, float]]] = []
505     work = [g]
506     while work:
507         h = work.pop()
508         if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509             pieces.append(h)
510             continue
511         split_i = TrackNetRunner._midpoint_split_index(h)
512         work.append(h[:split_i])
513         work.append(h[split_i:])
514         pieces.sort(key=lambda grp: grp[0][0])
515         out.extend(pieces)
516         flags.extend([True] * len(pieces))
517         forced += len(pieces) - 1
518     if forced:
519         logger.warning(
520             "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
521             "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522             "motion), not a bigger cap", forced)
523     return out, flags
524

```

5. assistant (2026-06-19T16:57:15.212Z)

Now let me look at the wasb variants and the FusionSegmenter to confirm the seam and the multi-track angle. Let me also check the wasb_runner since WASB may produce multiple tracks.

6. user (2026-06-19T16:57:16.247Z)

```

backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py

```

7. user (2026-06-19T16:57:16.247Z)


```
backend\pipeline\detectors\fusion_features.py
backend\pipeline\segmenters\fusion_hybrid.py
```

8. user (2026-06-19T16:57:19.467Z)

```
1      """WASB-based hybrid play segmenter.
2
3      Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5      badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7      For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8      candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9      min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
34             badminton "
35                 "weights) for precise candidate rallies + AI provider for semantic
36             boundaries.")
37
38         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
39             Any]]:
40             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
41             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
42
43     segmenter_registry.register(WasbHybridSegmenter)
```

9. user (2026-06-19T16:57:19.832Z)

```
1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4      1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5      2. copies wasb_infer.py into the WASB repo's src/ dir,
6      3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7      4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
```

```

13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45         # extraction that dominates a long clip. Output is bit-identical to the PNG path
46         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47         stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):

```

```

70         self.cfg = cfg
71
72     def healthcheck(self) -> Tuple[bool, str]:
73         """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74         cmd = (f"conda activate {self.cfg.conda_env} && "
75             f"python -c \"import torch; print('torch', torch.__version__, "
76             f"'cuda', torch.cuda.is_available())\"")
77         try:
78             res = self._wsl_bash(cmd)
79         except FileNotFoundError:
80             return False, "`wsl` not found ? Windows + WSL2 required."
81         except subprocess.TimeoutExpired:
82             return False, "WSL healthcheck timed out."
83         out = (res.stdout or "").strip()
84         if res.returncode != 0:
85             return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86         return True, out
87
88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104             res.stdout).strip())
105             return None
106         return dest
107
108     def _stage_infer_script(self) -> bool:
109         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110         src_mnt = to_wsl_mnt_path(_INFER_WIN)
111         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo "
112         OK"
113         res = self._wsl_bash(cmd)
114         return res.returncode == 0 and "OK" in (res.stdout or "")
115
116     def run_predict(self, video_win_path: str, output_win_dir: str,
117         video_id: Optional[str] = None) -> Optional[str]:
118         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122         can never reuse each other's staged frames, resumable cache, or CSV.
123
124         """
125         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
128         # the CSV landed inside WSL while Windows polled the repo-relative path.
129         output_win_dir = os.path.abspath(output_win_dir)
130         os.makedirs(output_win_dir, exist_ok=True)
131         # Normalize for cross-platform basename (tests use Windows paths on Linux).
132         video_win_path = str(video_win_path).replace("\\", "/")
133         base = os.path.basename(video_win_path)
134         stem, ext = os.path.splitext(base)
135         # Unique, content-derived key: same filename + different bytes -> different key.

```

```

132     key = f"{stem}__{video_id[:12]}" if video_id else stem
133     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136     if not self._stage_infer_script():
137         logger.error("Could not stage wasb_infer.py into WSL.")
138         return None
139     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140     if staged_video is None:
141         return None
142     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145     # WSL stage filesystem is already low so the user can clean up before it fills.
146     # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149         if free is not None and free < self.cfg.min_free_gb:
150             logger.warning(
151                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
152                 "may fill the disk ? consider running tools/cleanup_caches.py.",
153                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157     if self.cfg.stream_video:
158         frame_args = "--stream-video "
159     else:
160         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161     cmd = (
162         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163         f"python wasb_infer.py "
164         f"--video {wsl_tilde_quote(staged_video)} "
165         f"{frame_args}"
166         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167         f"--sport {shlex.quote(self.cfg.sport)} "
168         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
169     )
170     logger.info("Running WASB inference on %s ...", base)
171     try:
172         res = self._wsl_bash(cmd)
173     except subprocess.TimeoutExpired:
174         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175         return None
176     if res.returncode != 0:
177         logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178         return None
179     if not os.path.exists(expected_csv_win):
180         logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181         return None
182     logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184     # Success ? the trajectory CSV is the durable output; the staged video copy and
185     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187     # so the frame cache + detector cache remain for a resumable re-run.
188     # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189     if not self.cfg.keep_frames:
190         stale = [staged_video] if self.cfg.stream_video else [frames_dir,

```

```

staged_video]
191         logger.info("Cache hygiene: removing staged video%s for %s "
192                       "(set indexing.wasb.keep_frames=true to retain).",
193                       "" if self.cfg.stream_video else " + frame cache", key)
194         self._wsl_rm_rf(stale)
195         return expected_csv_win
196
197     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

10. assistant (2026-06-19T16:57:23.732Z)

Now let me check the FusionSegmenter to see how it overrides the seams, and the wasb_infer.py to confirm the CSV is a single point-stream (one X,Y per frame, no track IDs).

11. user (2026-06-19T16:57:24.682Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26    experiment harness (EXPERIMENT_HARNESS.md), never silently.
27    """
28    from typing import Any, Dict, List, Optional, Tuple
29
30    from backend.pipeline.segmenters.base import segmenter_registry
31    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32    from backend.pipeline.detectors.base import DetectorRunner
33    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig

```

```

35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
41 B)."""
42
43         family = "fusion"
44         display_label = "Fusion"
45
46         def __init__(self, db: Any, config: Any):
47             super().__init__(db, config)
48             # Built lazily only if the person cue is enabled (no detector model loaded
49 otherwise).
50             self._person: Optional[Any] = None
51             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
52 estimate it
53             # over the whole trajectory once per candidate window (it depends only on
54 `points`).
55             self._axis_cache_key: Optional[int] = None
56             self._axis_cache: Optional[tuple] = None
57
58         @property
59         def name(self) -> str:
60             return "fusion_hybrid"
61
62         @property
63         def description(self) -> str:
64             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
65 rallies, "
66                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
67 and the "
68                 "AI provider sets semantic boundaries.")
69
70         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
71 Any]]:
72             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
73             if substrate == "tracknet":
74                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
75                 return TrackNetRunner(cfg), {
76                     "weights_path": cfg.weights_path,
77                     "queue_length": cfg.queue_length,
78                     "fusion_substrate": "tracknet",
79                 }
80             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
81             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
82 "wasb"}
83
84         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
85 frame_width: float, video_path: str,
86 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
87             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
88 video_path, idx_cfg)
89             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
90 persisted
91             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
92 estimated
93             # over the whole trajectory by rally_gate (camera-agnostic); reused across
94 windows.
95             gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
96 min_crossings=0,
97 estimate=self._axis_estimate(points))
98             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
99
100

```

```

87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93         # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100        def _axis_estimate(self, points: List[Any]) -> tuple:
101            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102            computed once per video, not once per candidate window (it depends only on
points)."""
103            key = id(points)
104            if self._axis_cache_key != key or self._axis_cache is None:
105                self._axis_cache_key = key
106                self._axis_cache = rally_gate.estimate_play_axis(points)
107            est = self._axis_cache
108            assert est is not None
109            return est
110
111        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                             frame_width: float, video_path: str,
113                             fcfg: Dict[str, Any]) -> Dict[str, float]:
114            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115            frame range and compute the scale/translation-invariant person features. Lazily
builds
116            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118            from backend.pipeline.detectors.person_detector import PersonDetector
119            if self._person is None:
120                self._person = PersonDetector(self.config)
121
122            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127            fw = det["frame_width"] or frame_width
128            fh = det["frame_height"]
129            # If we can't trust the frame dims, every normalised feature would silently
collapse
130            # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131            if fw <= 0 or fh <= 0:
132                return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134            poly = fcfg.get("court_polygon")
135            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None

```

```

136         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                         court_polygon=polygon, net=net)
140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                           window_stats: List[Dict[str, Any]],
143                           idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
146         untouched (full superset).
147
148         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
149         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151         set rather than emit a video with zero rallies. This makes Gate A/B strictly
152         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154         below it always wins on an otherwise-empty video. Observed on real footage
155         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157         fcfg = idx_cfg.get("fusion", {})
158         min_crossings = int(fcfg.get("min_crossings", 0))
159         min_players = int(fcfg.get("min_players", 0))
160         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162         keep: List[int] = []
163         for i, st in enumerate(window_stats):
164             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                 continue
166             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                 continue
168             keep.append(i)
169
170         if not keep and window_stats:
171             # Gating emptied a non-empty video ? fall back to the ungated set.
172             return list(range(len(window_stats)))
173         return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177

```

12. user (2026-06-19T16:57:25.558Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8         Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.

```



```

13
14 Resumable across reboots
15 -----
16 The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17 the slow, expensive part; the CPU tracker pass that turns detections into a
18 trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19 resume mid-stream). We exploit that split:
20
21 * The detector pass writes its per-window raw outputs incrementally to
22 ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23 records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24 most ``--flush-every`` windows of GPU work instead of the whole pass.
25 * On restart we replay the cached windows, skip the DataLoader ahead to the first
26 un-cached window, and continue. Once every window is cached, the detector pass
27 is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28 the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29 not the GPU hour.
30
31 The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32 to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33 weights, sport, window size, or frame count change.
34
35 Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40
41 """
42 import os
43 import os.path as osp
44 import csv
45 import glob
46 import json
47 import argparse
48 import logging
49 from collections import defaultdict
50 from contextlib import contextmanager
51
52 logger = logging.getLogger(__name__)
53
54 CACHE_VERSION = 1
55 _DET_FILE = "det_raw.jsonl"
56 _MANIFEST_FILE = "manifest.json"
57
58 def _build_cfg(weights: str, sport: str):
59     """Compose the WASB eval config via Hydra, overriding model/weights/device."""
60     from hydra import compose, initialize
61     with initialize(config_path="configs", version_base=None):
62         cfg = compose(config_name="eval", overrides=[
63             f"dataset={sport}",
64             "model=wasb",
65             f"detector.model_path={weights}",
66             "runner.device=cuda",
67             "runner.gpus=[0]", # this box has a single GPU; default config assumes
68             ])
69     return cfg
70
71 def _frame_id(path: str) -> int:
72     """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
73     180)."""
74     stem = osp.splitext(osp.basename(path))[0]
75     digits = "".join(ch for ch in stem if ch.isdigit())

```

```

74         return int(digits) if digits else -1
75
76
77     # ----- #
78     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79     # ----- #
80     def _cache_paths(cache_dir: str):
81         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84     def default_cache_dir(out_csv: str) -> str:
85         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86         d = osp.dirname(osp.abspath(out_csv))
87         stem = osp.splitext(osp.basename(out_csv))[0]
88         return osp.join(d, stem + "_wasbcache")
89
90
91     def _atomic_write_text(path: str, text: str) -> None:
92         """Write then rename so a crash mid-write can't leave a half-written file."""
93         tmp = path + ".tmp"
94         with open(tmp, "w") as f:
95             f.write(text)
96             f.flush()
97             os.fsync(f.fileno())
98         os.replace(tmp, path)
99
100
101     def _det_plain(det) -> list:
102         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
103         list."""
104         xy = det["xy"]
105         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
106
107     def _dets_to_tracker(plain_list):
108         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
109         array,
110         the tracker does vector math / np.linalg.norm on it)."""
111         import numpy as np
112         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
113                 for p in plain_list]
114
115     def write_manifest(cache_dir: str, manifest: dict) -> None:
116         _, mpath = _cache_paths(cache_dir)
117         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120     def read_manifest(cache_dir: str):
121         _, mpath = _cache_paths(cache_dir)
122         if not osp.exists(mpath):
123             return None
124         try:
125             with open(mpath) as f:
126                 return json.load(f)
127         except (json.JSONDecodeError, OSError):
128             return None
129
130
131     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                             frames_in: int, frames_total: int) -> bool:
133         """A cache is reusable only if the run that produced it matches this run's
134         inputs."""
135         if not manifest or manifest.get("version") != CACHE_VERSION:
136             return False

```

```

136     return (
137         manifest.get("frames_dir") == osp.abspath(frames_dir)
138         and manifest.get("weights") == weights
139         and manifest.get("sport") == sport
140         and manifest.get("frames_in") == frames_in
141         and manifest.get("frames_total") == frames_total
142     )
143
144
145 def reset_cache(cache_dir: str) -> None:
146     """Drop any existing cache so the next run starts clean."""
147     det_jsonl, mpath = _cache_paths(cache_dir)
148     for p in (det_jsonl, mpath):
149         if osp.exists(p):
150             os.remove(p)
151
152
153 def load_and_clean_cache(cache_dir: str):
154     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156     prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159     [x, y, score, scale] candidates (accumulated across every window the frame is in,
160     in window order ? identical to a non-resumed run).
161     """
162     det_jsonl, _ = _cache_paths(cache_dir)
163     det_by_fid = defaultdict(list)
164     valid: list = []
165     if osp.exists(det_jsonl):
166         with open(det_jsonl, "r") as f:
167             for line in f:
168                 s = line.strip()
169                 if not s:
170                     continue
171                 try:
172                     obj = json.loads(s)
173                 except json.JSONDecodeError:
174                     break # truncated trailing line (crash mid-
flush)
175                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                     break
177                 valid.append(s)
178                 for fid, plain in obj["f"]:
179                     det_by_fid[int(fid)].extend([list(p) for p in plain])
180                 # Guarantee a clean append boundary by rewriting only the good prefix.
181                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182     return len(valid), det_by_fid
183
184
185 def _append_windows(fh, objs) -> None:
186     """Append window records as JSONL and durably flush (bounded loss on crash)."""
187     for o in objs:
188         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189     fh.flush()
190     os.fsync(fh.fileno())
191
192
193 def _atomic_write_trajectory(out_csv: str, rows) -> None:
194     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
195     tmp = out_csv + ".tmp"
196     with open(tmp, "w", newline="") as f:
197         w = csv.writer(f)
198         w.writerow(["Frame", "Visibility", "X", "Y"])
199         w.writerows(rows)

```

```

200         f.flush()
201         os.fsync(f.fileno())
202         os.replace(tmp, out_csv)
203
204
205         # ----- #
206         # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207         # ----- #
208         _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211     def synthetic_frame_path(index: int) -> str:
212         """Stable, index-encoding frame name used by the streaming path.
213
214         It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
215         downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216         frames came from disk or from the in-memory stream. The streaming image loader
217         inverts this name back to an index to fetch the decoded frame.
218         """
219         return _STREAM_FRAME_FMT.format(int(index))
220
221
222     def build_windows(frames: list, frames_in: int):
223         """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224         + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226         Returns a list of windows, each a list of ``frames_in`` consecutive paths
227         (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228         caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229         ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230         """
231         return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234     class SequentialFrameStore:
235         """In-memory sliding buffer over a forward-only frame reader, sized for the
236         detector's sliding-window access pattern (peak O(window) frames, not O(video)).
237
238         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
243         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
244         earliest index any not-yet-finished window can still ask for) and decode each source
245         frame exactly once via the forward-only ``reader(index)``.
246         """
247
248         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249             self._reader = reader
250             self._total = int(total)
251             self._window = max(1, int(window))
252             self._buf: dict = {}
253
254             # On a resumed run the detector's first needed frame is `start_index`; begin
255             # pulling there so the reader can skip (seek past) the already-cached prefix
256             # instead of decoding 0..start_index-1 just to throw them away.
257             self._next = int(start_index) # next index not yet pulled from the reader
258             self._max_seen = int(start_index) - 1
259             self._floor = int(start_index) # lowest index we still keep (never evict
260             below)
261
262         def get(self, index: int):
263             index = int(index)
264             if index < self._floor:
265                 raise KeyError(

```

```

264         f"frame {index} already evicted (below floor {self._floor}); the access
265         "
266         f"pattern must stay within a window of the highest frame seen")
267     if index >= self._total:
268         raise KeyError(f"frame {index} out of range (total {self._total})")
269     # Pull forward from the reader until the requested index is buffered.
270     while self._next <= index:
271         self._buf[self._next] = self._reader(self._next)
272         self._next += 1
273     if index > self._max_seen:
274         self._max_seen = index
275     # Evict frames older than the earliest a still-running window could re-request:
276     # once we've seen index `m`, no future window starts before `m - (window-1)`.
277     new_floor = max(self._floor, self._max_seen - (self._window - 1))
278     for k in range(self._floor, new_floor):
279         self._buf.pop(k, None)
280     self._floor = new_floor
281     return self._buf[index]
282
283 def _index_from_synthetic_path(path: str) -> int:
284     """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286     Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287     in lockstep with the on-disk naming scheme.
288     """
289     return _frame_id(path)
290
291
292 def _bgr_to_pil_rgb(frame_bgr):
293     """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
294     EXACT
295     PIL RGB image WASB's disk path produces.
296
297     The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
298     Image.open(PNG).convert('RGB')``;
299     because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
300     COLOR_BGR2RGB)``
301     bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
302     streamed
303     frame indistinguishable from the on-disk one to everything downstream.
304     """
305     import cv2
306     from PIL import Image
307     return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
308
309
310 def _make_streamed_read_image(store, real_read_image):
311     """Build the ``read_image`` replacement used during a streaming detector pass.
312
313     A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
314     image,
315     matching ``read_image``'s real return type); any path that doesn't parse to an index
316     falls
317     through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
318     testable
319     without the WSL-only ``dataloaders`` package.
320     """
321     def _read_image(path, *args, **kwargs):
322         idx = _index_from_synthetic_path(path)
323         if idx < 0:
324             return real_read_image(path, *args, **kwargs)
325         return _bgr_to_pil_rgb(store.get(idx))
326     return _read_image

```

```

321
322     @contextmanager
323     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
324         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
325         frames during the detector pass, byte-for-byte as it would over real PNGs.
326
327         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
328         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
329         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
330         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
331         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
332         ``osp.exists`` guard is sidestepped for synthetic stream paths.
333
334         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
335         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
336         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
337         reference).
338
339         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
340         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
341         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
342         original reader is restored on exit.
343         """
344         from dataloaders import dataset_loader as _dl
345         store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
346         real_read_image = _dl.read_image
347         # Rebind read_image in the consuming module for the duration of the detector pass;
348         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
349         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
350         try:
351             yield store
352         finally:
353             _dl.read_image = real_read_image # type: ignore[assignment]
354
355
356     # ----- #
357     # Inference
358     # ----- #
359     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
360             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361             log_every_batches: int = 50, cache_dir: "str | None" = None,
362             flush_every: int = 1000, fresh: bool = False,
363             frame_loader=None, source_key: "str | None" = None) -> str:
364         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366         Two equivalent ways to supply pixels (output is bit-identical between them):
367
368         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369         globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
370         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)

```

```

371     frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
372     pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
373     reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
374     ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
375     to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
376     ``cvtColor(bgr, BGR2RGB)`` , verified byte-identical). Streaming forces
``num_workers=0``
377     (the in-process shim + buffer can't cross worker processes).
378
379     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
380     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
381     after reboot still matches.
382     """
383     import time
384     import torch
385     from torch.utils.data import DataLoader
386     from detectors import build_detector
387     from trackers import build_tracker
388     from dataloaders import build_img_transforms, build_seq_transforms
389     from dataloaders.dataset_loader import ImageDataset
390     from utils import Center
391
392     streaming = frame_loader is not None
393
394     cfg = _build_cfg(weights, sport)
395     frames_in = int(cfg["model"]["frames_in"])
396     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
397     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
398
399     if streaming:
400         # `frames` is already the ordered list of (synthetic) frame paths.
401         if not isinstance(frames, (list, tuple)):
402             raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
403         frames = list(frames)
404         frames_dir = source_key or "stream"
405     else:
406         frames_dir = frames
407         frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
408         if limit:
409             frames = frames[:limit]
410         if len(frames) < frames_in:
411             where = frames_dir if not streaming else "video stream"
412             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
413
414         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
415         samples = []
416         for window in build_windows(frames, frames_in):
417             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
418             samples.append({"frames": window, "annos": annos})
419         windows_total = len(samples)
420
421         # ---- cache setup / resume decision ----- #
422         if cache_dir is None:
423             cache_dir = default_cache_dir(out_csv)
424             os.makedirs(cache_dir, exist_ok=True)
425             det_jsonl, _ = _cache_paths(cache_dir)
426             cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427             manifest = None if fresh else read_manifest(cache_dir)
428             resume = manifest_compatible(

```

```

429         manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430         frames_in=frames_in, frames_total=len(frames),
431     )
432     if resume:
433         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435     else:
436         if manifest is not None:
437             logger.info("cache incompatible with this run -> starting fresh")
438             reset_cache(cache_dir)
439             windows_done, det_by_fid = 0, defaultdict(list)
440
441     base_manifest = {
442         "version": CACHE_VERSION,
443         "frames_dir": cache_key,
444         "weights": weights,
445         "sport": sport,
446         "frames_in": frames_in,
447         "frames_total": len(frames),
448         "windows_total": windows_total,
449         "windows_done": windows_done,
450     }
451     write_manifest(cache_dir, base_manifest)
452
453     # ---- detector pass over the un-cached windows ----- #
454     remaining = samples[windows_done:]
455     if remaining:
456         _, transform_test = build_img_transforms(cfg)
457         try:
458             _, seq_transform_test = build_seq_transforms(cfg)
459         except Exception:
460             seq_transform_test = None
461         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462                         transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
463         # Streaming forces single-process loading: the in-memory frame store + the
464         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
465         loader_workers = 0 if streaming else num_workers
466         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
467         detector = build_detector(cfg)
468
469         from contextlib import ExitStack
470
471         t0 = time.time()
472         done = windows_done
473         win_idx = windows_done
474         log_every = max(1, log_every_batches)
475         flush_n = max(1, flush_every)
476         pending = []
477         logger.info(f"detecting on {len(remaining)} remaining windows "
478                 f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
479                 f"source={'video-stream' if streaming else 'frames-dir'})...")
480         det_f = open(det_jsonl, "a")
481         try:
482             with ExitStack() as stack:
483                 stack.enter_context(torch.no_grad())
484                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
485                 # in-memory decoded frame for that synthetic path; every other line of
486                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.

```



```

487         # The detector's first needed frame is `windows_done` (window i starts
at
488         # frame i), so prime the store there for a resume.
489         if streaming:
490             stack.enter_context(
491                 _streaming_read_image_patch(frame_loader, len(frames),
492                                             window=frames_in,
start_index=windows_done))
493         for bi, batch in enumerate(loader):
494             imgs, _hms, trans = batch[0], batch[1], batch[2]
495             img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
496             batch_results, _ = detector.run_tensor(imgs, trans)
497             nb = imgs.shape[0]
498             for ib in range(nb):
499                 frames_payload = []
500                 for ie in sorted(batch_results[ib].keys()):
501                     p = img_paths[ie][ib]
502                     fid = _frame_id(p)
503                     plain = [_det_plain(d) for d in batch_results[ib][ie]]
504                     frames_payload.append([fid, plain])
505                     det_by_fid[fid].extend(plain)
506                 pending.append({"w": win_idx, "f": frames_payload})
507                 win_idx += 1
508             done += nb
509             if len(pending) >= flush_n or done >= windows_total:
510                 _append_windows(det_f, pending)
511                 pending = []
512                 base_manifest["windows_done"] = done
513                 write_manifest(cache_dir, base_manifest)
514             if (bi + 1) % log_every == 0 or done >= windows_total:
515                 el = time.time() - t0
516                 new_done = done - windows_done
517                 rate = new_done / el if el > 0 else 0.0
518                 eta = (windows_total - done) / rate if rate > 0 else 0.0
519                 logger.info(f"{done}/{windows_total} "
520                             f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                             f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522             if pending:
523                 _append_windows(det_f, pending)
524                 base_manifest["windows_done"] = done
525                 write_manifest(cache_dir, base_manifest)
526         finally:
527             det_f.close()
528             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529     else:
530         logger.info("detector cache already complete -> tracker-only re-run")
531
532     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533     tracker = build_tracker(cfg)
534     tracker.refresh()
535     rows = []
536     for fid in sorted(det_by_fid.keys()):
537         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540     _atomic_write_trajectory(out_csv, rows)
541     visible = sum(1 for _, v, *_ in rows if v)
542     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
543     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544     return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:

```

```

548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:
568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578     def _probe_frame_count(video_path: str) -> int:
579         """Container-reported frame count via cv2 (0 if unknown). Used to size the
580         stream."""
581         import cv2
582         cap = cv2.VideoCapture(video_path)
583         if not cap.isOpened():
584             raise SystemExit(f"cannot open video: {video_path}")
585         n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586         cap.release()
587         return n
588
589     def make_video_frame_loader(video_path: str):
590         """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
591
592         ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
593         long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594         detector's sliding-window order); it reads sequentially and only uses a frame-seek
595         to
596         skip forward when a *resume* starts past the current position. Each ``cap.read()``
597         returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
598         would
599         yield (PNG is lossless), so the detector sees identical pixels to the disk path.
600
601         Returns the count hint from the container so the caller can build the frame list;
602         ``release`` closes the capture.
603         """
604         import cv2
605         cap = cv2.VideoCapture(video_path)
606         if not cap.isOpened():
607             raise SystemExit(f"cannot open video: {video_path}")
608         count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
609         state = {"pos": 0} # next index cap.read() will return
610
611         def frame_loader(index: int):

```

```

610         index = int(index)
611         if index < state["pos"]:
612             raise RuntimeError(
613                 f"streaming decode is forward-only; got index {index} after
{state['pos']}")
614         if index > state["pos"]:
615             # Forward skip only happens when resuming past cached windows. Seek once.
616             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
617             state["pos"] = index
618             ok, frame = cap.read()
619             if not ok:
620                 raise RuntimeError(f"failed to decode frame {index} from {video_path}")
621             state["pos"] += 1
622             return frame
623
624     def release():
625         cap.release()
626
627     return frame_loader, count_hint, release
628
629
630     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                             limit: int = 0, batch_size: int = 8,
632                             log_every_batches: int = 50, cache_dir: "str | None" = None,
633                             flush_every: int = 1000, fresh: bool = False) -> str:
634         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637         ``run``'s docstring for why.
638         """
639         frame_loader, count_hint, release = make_video_frame_loader(video_path)
640         try:
641             if count_hint <= 0:
642                 raise SystemExit(
643                     f"could not determine frame count for {video_path}; cannot stream "
644                     f"(fall back to frame extraction with --frames_dir)")
645             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646             source_key = "video:" + osp.abspath(video_path)
647             return run(frame_list, weights, out_csv, sport, limit,
648                       batch_size=batch_size, num_workers=0,
649                       log_every_batches=log_every_batches, cache_dir=cache_dir,
650                       flush_every=flush_every, fresh=fresh,
651                       frame_loader=frame_loader, source_key=source_key)
652         finally:
653             release()
654
655
656     def main():
657         ap = argparse.ArgumentParser()
658         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
660         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661         ap.add_argument("--stream-video", action="store_true",
662                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
663                             "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664         ap.add_argument("--weights", required=True)
665         ap.add_argument("--out", required=True)
666         ap.add_argument("--sport", default="badminton")
667         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")

```

```

668     ap.add_argument("--batch-size", type=int, default=8)
669     ap.add_argument("--num-workers", type=int, default=4)
670     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671     ap.add_argument("--cache-dir", default=None,
672                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673     ap.add_argument("--flush-every", type=int, default=1000,
674                     help="fsync the detector cache every N windows (bounds reboot
loss)")
675     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676     args = ap.parse_args()
677
678     if args.stream_video:
679         if not args.video:
680             raise SystemExit("--stream-video requires --video")
681         run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                             batch_size=args.batch_size,
683                             log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                             flush_every=args.flush_every, fresh=args.fresh)
685     return
686
687     frames_dir = args.frames_dir
688     if args.video:
689         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690     if not frames_dir:
691         raise SystemExit("provide --frames_dir or --video")
692     run(frames_dir, args.weights, args.out, args.sport, args.limit,
693         batch_size=args.batch_size, num_workers=args.num_workers,
694         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695         flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700

```

13. assistant (2026-06-19T16:57:29.608Z)

I now have the full picture. Let me verify the key data-shape claim: the tracker emits exactly one (x,y) per frame (no track IDs), confirming the CSV is a single point-stream. I've confirmed this at wasb_infer.py lines 536-538 (one `r = tracker.update(...)` per frame_id ? one row). Let me also confirm there is no multi-track selection layer anywhere by checking the registry default and rally_gate briefly.

14. user (2026-06-19T16:57:31.266Z)

```

backend\pipeline\detectors\person_detector.py:15:    with the motion stats
(`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
backend\pipeline\detectors\person_detector.py:104:    Returns a list of (track_id, bbox)
where bbox is (x1, y1, x2, y2) in pixels.
backend\pipeline\detectors\person_detector.py:148:    track_id_count, chunk_index.
backend\pipeline\detectors\person_detector.py:176:    # Per active frame we keep (frame_idx,
peak_velocity, set_of_active_track_ids)
backend\pipeline\detectors\person_detector.py:200:    for track_id, bbox in detections:
backend\pipeline\detectors\person_detector.py:201:        current_ids.add(track_id)
backend\pipeline\detectors\person_detector.py:203:        if track_id in track_history:
backend\pipeline\detectors\person_detector.py:204:            prev_bbox =
track_history[track_id]
backend\pipeline\detectors\person_detector.py:208:        frame_active_ids.add(track_id)
backend\pipeline\detectors\person_detector.py:210:            track_history[track_id] = bbox

```

```

backend\pipeline\detectors\person_detector.py:231:         track_ids = set().union(*[ids for
_, _, ids in group])
backend\pipeline\detectors\person_detector.py:238:         "track_id_count":
len(track_ids),
backend\pipeline\detectors\person_detector.py:270:         Returns ``{"frames": {frame_idx:
[(track_id, bbox), ...]}, "fps", "frame_width",
backend\pipeline\detectors\fusion_features.py:11:- A person box is ``(x1, y1, x2, y2)`` in
**pixels**; ``(track_id, box)`` when motion needs it.
backend\pipeline\detectors\fusion_features.py:12:- ``per_frame`` is ``{frame_idx: [(track_id,
bbox), ...]}`` keyed by the ORIGINAL video frame
backend\pipeline\detectors\tracknet_runner.py:355:         "track_id_count": 1, # single
shuttle; kept for window-shape parity
backend\pipeline\segmenters\trajectory_hybrid.py:93:         "track_id_count":
cw["track_id_count"],
backend\pipeline\segmenters\yolo_hybrid.py:200:         "track_id_count":
max(a["track_id_count"], b["track_id_count"]),
backend\pipeline\segmenters\yolo_hybrid.py:233:         f"tracks={w['track_id_count']}"
backend\pipeline\segmenters\yolo_hybrid.py:245:         "track_id_count":
w["track_id_count"],

```

15. assistant (2026-06-19T16:57:34.794Z)

Confirmed. The shuttle path has no track-ID concept (`track\_id\_count` is hardcoded to 1 at tracknet_runner.py:355); track IDs only exist in the person/YOLO paths. The WASB tracker emits exactly one point per frame. I have everything I need to return the verdict.

16. user (2026-06-19T16:57:55.945Z)

Structured output provided successfully

17. assistant (2026-06-19T16:58:07.154Z)

I've traced the production path and returned the structured verdict. Summary of findings:

```

**The path** (`backend/pipeline/segmenters/trajectory_hybrid.py::process_video`):
- Line 197: `runner.run_predict(...)` ? one trajectory CSV
- Line 202: `runner.parse_trajectory_csv(csv_path)` ? one flat `List[TrajectoryPoint]`
- Lines 206-217: `runner.trajectory_to_action_windows(points, ...)` ? candidate windows

```

```

**The data shape is single-track by construction.** The WASB tracker emits exactly one `(x,y)`
per frame (`wasb_infer.py:536-538`), the CSV schema is `Frame,Visibility,X,Y`
(`wasb_infer.py:198-202`), and `parse_trajectory_csv` returns a flat point list with no track
dimension (`tracknet_runner.py:237-253`). The windower hardcodes `"track_id_count": 1`
(`tracknet_runner.py:355`).

```

```

**The exact injection seam** is `trajectory_hybrid.py:202 ? 206` (between parse and windowing).
I marked the multi-track claim **PARTIAL** because:
- A *selected-track* layer is a clean drop-in ? the windower's contract is just
`List[TrajectoryPoint]`, so a layer that picks one track from per-track data and emits a point
list slots in with the windower unchanged.
- A *per-track-window union* is feasible but not free: the windower ingests only a point stream
(not pre-built windows), so unioning means calling it per-track and merging window lists
*outside* it. And critically, **nothing upstream currently produces multiple tracks** ? both the
tracker output and the CSV schema would need to carry per-track identity first.

```

Relevant files (all absolute):

```

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\tracknet_runner.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb_runner.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb_infer.py`

```

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\fusion\_hybrid.py`